I did not order this! Why is it on my bill?

Document Number: P1190R0
Date: 2018-08-06
Author: David Stone (davidmstone@google.com, david@doublewise.net)
Audience: LEWG, EWG

Purpose

There are many types in the standard library that have comparison operators that defer to the comparison operators of some template parameter (for instance, `vector`, `tuple`, and `variant`, henceforth "wrappers"). This paper explores the issues surrounding `operator<=>` for such wrappers. In particular, for some types, there is an important reason to define `operator==` and `operator!=` even in a world with `operator<=>`: it is sometimes cheaper to determine that two values are not equal than it is to determine in which way they are unequal. The canonical example here is differently-sized containers. Comparing their sizes is cheap (single integer comparison that does not need to follow a pointer), but comparing a value could be arbitrarily expensive. The current behavior of containers like `vector` is that no comparisons are performed on the contained objects if the sizes are not equal. It would be an unfortunate pessimization if `operator==` on a `vector<int>`, `vector<vector<int>>`, `tuple<vector<int>>`, or `struct { vector<int>; }` suddenly got slower in C++20. This paper explores several potential solutions to decide how to specify `operator<=>` for wrapper types.

Prior work

- N4126: "Defaulted comparison operators", Oleg Smolsky: This paper proposed supporting `= default` on all comparison operators and had no rules for implicit generation. It got wide support, but was eventually withdrawn in favor of P0221R2. The primary objection to this paper was the verbosity of enabling comparisons.
- P0221: "Default comparisons", Bjarne Stroustrup: This paper proposed generating all comparison operators by default if certain conditions were met. This made it to a full committee vote in plenary, during which it was rejected. The primary objections to this paper were that `operator<` should not be generated by default, certain relational operators were not as performant as they could be, and there was no way to opt-out of the behavior.
- P0481: "Bravely Default", Tony Van Eerd: This paper proposed generating only `operator==` and `operator!=` by default if the copy constructor is implicitly generated, with explicit syntax of `= default` and `= delete` supported when necessary. This paper got a reasonable level of support in terms of the direction proposed. Ultimately, we abandoned this approach in favor of `operator<=>`. The primary objection to this paper was that it did not attempt to completely solve the comparison problem, it solved it only for equality.
- P0432: "Implicit and Explicit Default Comparison Operators", David Stone: This paper proposed much of the same mechanisms as P0481R0, but it also supported `= default` for relational operators (<, <=, >, and >=). "Something kind of like this" was well supported. This paper was withdrawn in favor of `operator<=>`. The primary objections to this paper were that it did not help with the code duplication between comparison operators and it led to less efficient relational operators.
- P0515: "Consistent comparison" (`operator<=>`), Herb Sutter: This paper was a synthesis of many of the previously mentioned paper (plus several others), in addition to new material. This paper was voted into the standard.
- P0768: "Library Support for the Spaceship (Comparison) Operator", Walter E. Brown: This is the companion paper to P0515 and describes the additions to the standard library needed to support `operator<=>`.
- P0790: "Effect of operator<=> on the standard library", David Stone: This paper discusses simple additions of `operator<=>` to the standard library.

What do we value?

- **Performance**: "You don't pay for what you don't use". This is the guiding principle of C++. The abstractions provided by the language should be at least as efficient as those that a programmer could code manually. We should not impose unnecessary overhead.
- **Maintainability**: "Don't repeat yourself". We want to specify a particular behavior in exactly one place.
- **Respect the programmer's time**: "Make easy things easy". If the language can make a decision that is always (or almost always) correct, then we should not have to do it manually. If there is only one way to implement a feature, we should not require code to say *how* to do it.
- **Respect the programmer's choices**: "Make hard things possible". Conversely, if there are many trade-offs involved and we cannot make one decision to fit everyone, we should refrain from making that decision for everyone.
- **Backward compatibility**: The more existing code we break, the stronger justification we need. It is better to cause existing code to fail to compile rather than silently change behavior.

The Problems

Performance

The obvious specifications for `operator<=>` on variable-sized containers is significantly slower than `operator==` is today because `operator<=>` cannot short circuit based on size (you know the two containers are unequal, but you do not know which is less), but `operator==` can. This ends up having far reaching consequences.

Backward compatibility

When we add `operator<=>` to wrapper types, what do we do when the wrappers are given types that do not implement `operator<=>`? If we define the wrapper's `operator<=>` to call the wrapped type's `operator<=>`, we do not generate `operator<=>` for such a type, and if we unconditionally remove `operator==`, `operator!=`, `operator<`, `operator<=`, `operator>`, and `operator>=` from the wrapper types, the wrappers suddenly become non-comparable when instantiated with such types. If we unconditionally use `compare_3way` to implement these types so that we fall back on `operator==` and `operator<` for the underlying type, we end up with a silent performance change that makes such types up to twice as expensive to compare.

Background

Prior to `operator<=>`, how do you implement `operator==` for a `SequenceContainer`, such as `vector`? (note: this paper intentially inlines the call to algorithms like `equal` and `lexicographical_compare`, and names the functions something other than `operator==` to give them a name for discussion)

```
template<typename T>
bool equal1(vector<T> const & lhs, vector<T> const & rhs) {
    if (lhs.size() != rhs.size()) {
        return false;
    }
    auto lhs_first = lhs.begin();
    auto const lhs_last = lhs.end();
    auto rhs_first = rhs.begin();
    while (lhs_first != lhs_last) {
        if (!(*lhs_first == *rhs_first)) {
            return false;
        }
        ++lhs_first;
        ++rhs_first;
```

```
    }
    return true;
}
```

This ensures that two differently-sized containers will immediately return false. It performs a single comparison of a data member within the `vector`, then each iteration performs a single iterator comparison. It is important to note that the early size check lets us avoid following the internal data pointer in the container and lets us avoid a loop / function call. If we try to implement this without having the size check (note that it is currently mandated that implementations perform this size check), we have two possible solutions:

```
template<typename T>
bool equal2(vector<T> const & lhs, vector<T> const & rhs) {
    auto lhs_first = lhs.begin();
    auto const lhs_last = lhs.end();
    auto rhs_first = rhs.begin();
    auto const rhs_last = rhs.end();
    while (lhs_first != lhs_last && rhs_first != rhs_last) {
        if (!(*lhs_first == *rhs_first)) {
            return false;
        }
        ++lhs_first;
        ++rhs_first;
    }
    return lhs.size() == rhs.size();
}

template<typename T>
bool equal3(vector<T> const & lhs, vector<T> const & rhs) {
    auto impl = [](vector<T> const & smaller, vector<T> const & larger) {
        auto smaller_first = smaller.begin();
        auto const smaller_last = smaller.end();
        auto larger_first = larger.begin();
        while (smaller_first != smaller_last) {
            if (!(*smaller_first == *larger_first)) {
                return false;
            }
            ++smaller_first;
            ++larger_first;
        }
        return larger_first == larger.end();
    };

    return lhs.size() <= rhs.size() ? impl(lhs, rhs) : impl(rhs, lhs);
}
```

You can see the generated code for these versions on Godbolt.

`equal2` is the straightforward approach, which ignores the problem. It means that for each element in the smaller container, we have two iterator comparisons instead of just one. For the common case of `vector<Integer>` or `string`, this changes our per iteration cost from two increments and two compare + branch to two increments, one compare + branch, and one compare + compare + and + branch. Using a very rough estimation, this adds an extra two operations per loop.

`equal3` tries to be smarter. It is optimized for longer sequences, because it pays a small fixed cost for the whole comparison to keep the loop costs the same as the first version. Something much like this is essentially what gcc's libstdc++ did for comparing `string` with `char const *`. Compared with `equal1`, this was benchmarked to dramatically increase the costs of comparisons (depending on the benchmark, anywhere from 0 to 66% of the total cost was removed when moving from something like `equal3` to something like `equal1`, but with `strlen` and `memcmp` mixed in).

Neither of these two options are satisfying. They all have at least as much in fixed costs as `equal1`, and `equal2` has much more in per-loop costs. Unfortunately, straightforward application of `operator<=>` to the containers gives us a generated `operator==` that is the moral equivalent to `equal2` or `equal3`.

Order! Order in the container!

`operator<=>` for containers must support ordering containers if the contained type can be ordered. If we want to maintain current ordering (more on that later), `operator<=>` would look something one of these two solutions:

```
// Does something like `equal2` above
template<typename T>
auto compare2(vector<T> const & lhs, vector<T> const & rhs) {
    auto lhs_first = lhs.begin();
    auto const lhs_last = lhs.end();
    auto rhs_first = rhs.begin();
    auto const rhs_last = rhs.end();
    while (lhs_first != lhs_last && rhs_first != rhs_last) {
        if (auto const cmp = compare_3way(*lhs_first, *rhs_first); cmp != 0) {
            return cmp;
        }
        ++lhs_first;
        ++rhs_first;
    }
    return lhs.size() <=> rhs.size();
}

// Does something like `equal3` above
template<typename T>
auto compare3(vector<T> const & lhs, vector<T> const & rhs) {
    auto impl = [](vector<T> const & smaller, vector<T> const & larger) {
        auto smaller_first = smaller.begin();
        auto const smaller_last = smaller.end();
        auto larger_first = larger.begin();
        while (smaller_first != smaller_last) {
            if (auto const cmp = compare_3way(*smaller_first, *larger_first); cmp != 0) {
                return cmp;
            }
            ++smaller_first;
            ++larger_first;
        }
        return larger_first == larger.end() ? strong_ordering::equal : strong_ordering::less;
    };
    auto reverse_ordering = [](auto order) {
        return 0 <=> order;
    };

    return lhs.size() <= rhs.size() ? impl(lhs, rhs) : reverse_ordering(impl(rhs, lhs));
}
```

Scope of the problem

Ranges that are affected by this problem

These types are all variable-size, sized, ordered ranges, which are all directly affected by this problem.

- `basic_string`
- `basic_string_view`
- `deque`
- `list`
- `vector` (including `vector<bool>`)

- `map`
- `set`
- `multimap`
- `multiset`
- `queue`
- `stack`

Types that wrap another type, and thus could stumble onto this problem if we do nothing

- `array`
- `pair`
- `tuple`
- `reverse_iterator`
- `move_iterator`
- `optional`, including heterogeneous `optional<T> <=> optional<U>`, `optional<T> <=> U`, `optional<T> <=> nullopt_t`, and `nullopt_t <=> nullopt_t`
- `variant`
- `forward_list`

All of these are either fixed-size (in many cases, size == 1) or, in the case of `forward_list`, unsized, and thus would not benefit from an early exit based on different sizes. If we can define a maximally efficient `operator<=>` for all types, these types don't need to do anything special with regards to performance (`operator<=> = default` would be sufficient).

Containers that are not affected by this performance problem

- `unordered_map`
- `unordered_set`
- `unordered_multimap`
- `unordered_multiset`

The containers on this list provide only `operator==` and `operator!=`. Their `operator<=>` will return, at best, `strong_equality`. As long as a valid implementation of `operator<=>` is to call `a == b` for each element, these containers are unaffected by the performance problem outlined in this paper.

Possible solutions for the performance problem

Define `operator==` and `operator!=` for ranges

This is a tempting solution. The author of the range type merely defines their own `operator==` (and `operator!=` in terms of it) that does the size short circuiting. Unfortunately, this solution just pushes the problem up. If we define `operator==` and `operator!=` for ranges, we run into the same problem with the obvious specification of `operator<=>` for types like `tuple`, so now any type that calls another type's `operator<=>` also has to define its own `operator==` and `operator!=`. Even if we specify types like `tuple` to have all three of `operator<=>`, `operator==`, and `operator!=`, current wording for `operator<=>` would make user-defined types like

```
struct S {
    string str;
    auto operator<=>(S const &) const = default;
};
```

slower than `string`. If we accept this library-only solution, we must sacrifice maintainability and respecting the programmer's time.

Define `operator==` and `operator!=` for ranges, and synthesize those operators in general

This takes the previous approach, but does not simply define defaulted `operator==` and `operator!=` in terms of `operator<=>`. Instead, we create a slightly more complicated set of rules to pick up user-defined `operator==`.

- For the expression `a != b`, call `!(a == b)` instead of `a <=> b != 0`. For types that define only `operator<=>` this has the same effect as current rules.
- When the user defines `auto operator<=>(T const &) const = default;`, this gives a memberwise `operator<=>` and a memberwise `operator==`. In other words, rather than creating a rewrite rule for `operator==`, we define an actual operator. For types that contain only data members that do not have an explicitly-defined `operator==`, this has the same effect as the current rules. If, however, some of the data members do have an explicitly-defined `operator==`, this will ensure that the generated `operator==` and `operator!=` for the wrapper respect the definition of each contained type. This will ensure that `tuple<string>` and the `struct S` example above naturally are as efficient as possible when compared for equality.

There are three main issues with this approach:

- Increased code size. Rather than having one function defined that all other operators are rewritten to be in terms of, we have two. In practice, this would be necessary only in those cases where the user has defined `operator==` (presumably because it was necessary), so this is an example of paying for what you do use.
- Defaulting `operator<=>` is also defaulting `operator==`. It's a bit strange that a user would have to define both `operator<=>` and `operator==` to get the behavior they would get from `auto operator<=>(...) const = default`. We could make this behavior more explicit and require users to type `bool operator== (T const &) const = default` to opt-in to this behavior, but I do not recommend this approach because forgetting to do so leads to a silent performance problem, rather than a compiler error. We could make it a compiler error to use `operator==` on a type that has `auto operator<=>(...) const = default` if any of the data members have an explicitly-defined `operator==`. At this point, we are making the user type more code to get the behavior that they almost certainly want (what would the error message from your compiler look like? "You meant to put `bool operator==(T const &) const = default;` somewhere in your class."). The model of `operator<=>` is already that it is rewriting calls to other operators, so from that perspective it might not be that strange.
- We need to define when exactly we follow the rule of "For the expression `a != b`, call `!(a == b)`". There are two options here: 1) If `a == b` is valid (changes code without `operator<=>`) or 2) Only when `operator<=>` is defined (we change the specification of `operator<=>` to rewrite `a != b` as `!(a == b)` instead of `a <=> b != 0`).

Make `operator<=>` create only `operator<`, `operator<=`, `operator>`, and `operator>=`

In other words, `operator<=>` would be for ordering only, not equality. If we take this approach, we would also want to allow `bool operator==(T const &) const = default;` and `bool operator!=(T const &) const = default;`. Where the above approach causes a silent performance issue when we try to make this behavior explicit, this approach simply does not give you equality unless you explicitly ask for it.

There are two main issues with this approach:

1) Compared to the previous solution, this requires the user to type even more to opt-in to behavior that they almost always want (if you have defaulted relational operators, you probably want the equality operators). Because `operator<=>` is a new feature, we do not have any concerns of legacy code, so if the feature starts out as giving users all six comparison operators, it would be better if they must type only one line rather than having to type three. 2) It is a natural side-effect of computing less than vs. greater than that you compute equal to. It is strange that we define an operator that can tell us whether things are equal, but we use it to generate all comparisons other than equal and not equal. For the large set of types for which `operator<=>` alone is sufficient, it also means that users who are not using the default (they are explicitly defining the comparisons) must define

two operators that encode much of the same logic of comparison. This mandatory duplication invites bugs as the code is changed under maintenance.

Add a new comparison category that creates only `operator<`, `operator<=`, `operator>`, and `operator>=`

An interesting modification to this idea would be to create one more comparison category that, when used as the return type of `operator<=>`, leads to only the relational operators being synthesized. When calling `operator<=>` directly, there would still be a value that tells whether the two objects are equal (as this is a natural side-effect), it would be only the generation that is affected.

This has most of the same drawbacks as the previous approach, except that it allows the user to say "No, this type is different, `operator<=>` is not good enough for `operator==` and `operator!=`".

Pascalexicographical order

The previous solutions accept that `operator<=>` is slower than `operator==` and provide ways to have both. The ideal solution would make `lhs <=> rhs == 0` be just as fast as `lhs == rhs`, and have `lhs == rhs` be just as fast in C++20 as it is in C++17. We can have such a solution if we are willing to sacrifice either backward compatibility or consistency. In C++17 and earlier, `operator<` gives a lexicographical order for containers, which is an extension of alphabetical order. It essentially sorts first by elements, then by size. "Pascalexicographical order" sorts first by size, then by elements.

Lexicographical (what we currently have)	Pascalexicographical
`"a"s == "a"s`	`"a"s == "a"s`
`"a"s < "z"s`	`"a"s < "z"s`
`"a"s < "aa"s`	`"a"s < "aa"s`
`"aa"s < "z"s`	`"z"s < "aa"s`

What does an `operator<=>` that gives this ordering look like?

```
template<typename T>
auto compare1(vector<T> const & lhs, vector<T> const & rhs) {
    if (auto const cmp = lhs.size() <=> rhs.size(); cmp != 0) {
        return cmp;
    }
    auto lhs_first = lhs.begin();
    auto const lhs_last = lhs.end();
    auto rhs_first = rhs.begin();
    while (lhs_first != lhs_last) {
        if (auto const cmp = compare_3way(*lhs_first, *rhs_first); cmp != 0) {
            return cmp;
        }
        ++lhs_first;
        ++rhs_first;
    }
    return strong_ordering_equal;
}
```

Now we are back to essentially the same fast code that we had for `equal1`.

This operator fully embraces the idea that for many uses, "any order" is sufficient, so it provides the fastest strong ordering possible. There are certainly users that depend on the order of these wrappers (that is to say, they need the specific order given by a lexicographical comparison, they do not just need any ordering). This means that the straightforward application of this change would be a major, silent breaking change. But is there some clever way we can do this such that it does not break any users?

One thought could be to define `operator<=>` with this new order only for those types that, themselves, define `operator<=>`. We know that no user code is doing this today, so no existing code will change behavior. This immediately runs into one major stumbling block, the foremost example of which is `basic_string<char>`. `char` has `operator<=>`, and changing the default sorting order of strings is likely the biggest breaking change in this category. If we try to exclude `basic_string<char>` from this change in ordering, we also lose out on most of the performance benefits.

The other option would be to make this change for `operator<=>` only. The long-term plan could be for `operator==` and `operator!=` to not be defined explicitly for these wrapper types (they just defer to `operator<=>`), but the relational operators remain and provide a lexicographical ordering. This maintains backward compatibility and ensures that we generate code that is fast by default, but has a huge loss in consistency (for all of the most common standard library types, `a <=> b < 0` is not the same as `a < b`, despite that being the rewrite rule).

In theory, there could exist a tool that (as part of the C++20 upgrade process) rewrote all existing uses of relational operators like `operator<` to be `lexicographical_compare` for affected containers...

How to specify wrapper types (other than the performance problem)

Any solution we decide on (other than the non-solution of not providing `operator<=>` for any of these types) will have at least one thing in common: the return type will be the common comparison result type for comparing each of the elements, similar to `lexicographical_compare_3way`.

When given a type that does not have `operator<=>`, there are two distinct possibilities: either we provide `operator<=>` or we do not. The standard library has `compare_3way`, which calls `operator<=>` if possible, otherwise it synthesizes it from `operator==` and `operator<`. If we require all calls in wrapper types to go through `compare_3way` rather than `operator<=>` on all wrapped types, we get `operator<=>` automatically for free. This would also work with the default `lexicographical_compare_3way` as an implementation strategy. It seems better to always provide `operator<=>` to give a consistent user experience ("`vector<T>` always has `operator<=>`") and it can give better performance in situations that require 3-way comparisons -- in the case of `vector<vector<T>>`, for instance, doing all necessary comparisons on the first `vector` element, even if that implementation is synthesized, means that you iterate over the outer vector only once, which is slightly less traffic on your memory bus.

However, we would not want to remove all of the old comparison operators for types that do not define `operator<=>`, otherwise we would have a silent performance degredation from C++17 to C++20. Here, we have a choice again. We can define the 6 comparison operators only for those types that do not define `operator<=>`, or we can define them always. Here, it probably makes sense to define them only for types that do not have `operator<=>`, unless we go for a solution to the performance problem that involves generating `operator==` and `operator!=`, in which case we should define those as well.

Summary / Suggested polls

- Synthesize a memberwise `operator==` when the user defaults `operator<=>`
- Synthesize `operator!=` as being rewritten to `!(lhs == rhs)` (following the same precedence / name hiding rules as `operator<=>`) for all types
- Synthesize `operator!=` as being rewritten to `!(lhs == rhs)` (following the same precedence / name hiding rules as `operator<=>`) only when `operator<=>` is defined
- Make `operator<=>` create only `operator<`, `operator<=`, `operator>`, and `operator>=`, not `operator==` or `operator!=`
- Add a new comparison category that creates only `operator<`, `operator<=`, `operator>`, and `operator>=`, not `operator==` or `operator!=`
- Change the way all containers are ordered with all comparison operators to be implementation-defined

- Change the way all containers are ordered with all comparison operators to be "Pascalexicographical", thus not comparing any contained elements if the sizes are unequal
- Change the way all containers are ordered with `operator<=>` only to be implementation-defined
- Change the way all containers are ordered with `operator<=>` only to be "Pascalexicographical", thus not comparing any contained elements if the sizes are unequal
- Wrapper types should always have `operator<=>`, even when given a type that does not have `operator<=>`
- Deprecate the comparison operators from wrapper types if the wrapped type has `operator<=>`
- Remove the comparison operators from wrapper types if the wrapped type has `operator<=>`